

[Home](#)[GitHub](#)

CLASSES

[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)[Footer](#)[GET /](#)

frontend/src/utils/formatters.js

```
1.  /**
2.   * @file Funciones de utilidad para formatear fechas.
3.   * @description Contiene funciones para formatear fechas en diferentes formatos.
4.   */
5.
6.  /**
7.   * @function formatDate
8.   * @description Formatea una fecha a un formato largo y legible.
9.   * @param {string|Date} date - La fecha a formatear.
10.  * @returns {string} La fecha formateada (e.g., "2 de diciembre de 2023").
11.  */
12. export const formatDate = (date) => {
13.   if (!date) return '';
14.
15.   const dateObj = new Date(date);
16.
17.   // Verificar si es fecha válida
18.   if (isNaN(dateObj.getTime())) return '';
19.
20.   const options = {
21.     year: 'numeric',
22.     month: 'long',
23.     day: 'numeric',
24.     hour: '2-digit',
25.     minute: '2-digit'
26.   };
27.
28.   return dateObj.toLocaleDateString('es-ES', options);
29. };
30.
31. /**
32.  * @function formatDateShort
33.  * @description Formatea una fecha a un formato corto (DD/MM/YYYY).
34.  * @param {string|Date} date - La fecha a formatear.
35.  * @returns {string} La fecha formateada en formato corto.
36.  */
37. export const formatDateShort = (date) => {
38.   if (!date) return '';
39.
40.   const dateObj = new Date(date);
41.
42.   if (isNaN(dateObj.getTime())) return '';
43.
44.   const day = String(dateObj.getDate()).padStart(2, '0');
45.   const month = String(dateObj.getMonth() + 1).padStart(2, '0');
46.   const year = dateObj.getFullYear();
47.
48.   return `${day}/${month}/${year}`;
49. };
50.
51. /**
52.  * @function timeAgo
```

```

53.  * @description Calcula y formatea el tiempo transcurrido desde una
54.  * @param {string|Date} date - La fecha desde la que calcular el tie
55.  * @returns {string} Una cadena que representa el tiempo transcurrido
56.  */
57.  export const timeAgo = (date) => {
58.      if (!date) return '';
59.
60.      const dateObj = new Date(date);
61.      if (isNaN(dateObj.getTime())) return '';
62.
63.      const seconds = Math.floor((new Date() - dateObj) / 1000);
64.
65.      const intervals = {
66.          año: 31536000,
67.          mes: 2592000,
68.          semana: 604800,
69.          día: 86400,
70.          hora: 3600,
71.          minuto: 60,
72.          segundo: 1
73.      };
74.
75.      for (const [name, secondsInInterval] of Object.entries(intervals)) {
76.          const interval = Math.floor(seconds / secondsInInterval);
77.
78.          if (interval >= 1) {
79.              return `Hace ${interval} ${name}${interval > 1 ? 's' : ''}`;
80.          }
81.      }
82.
83.      return 'Justo ahora';
84.  };

```

Documentation generated by [JSDoc 4.0.5](#) on Thu Dec 11 2025 08:15:36 GMT+0000
(Coordinated Universal Time) using the [docdash](#) theme.